

Smart Contract Assessment

Moonwell

HALBORN

Smart Contract Assessment - Moonwell

Prepared by:  HALBORN

Last Updated 02/13/2026

Date of Engagement: January 20th, 2026 - February 4th, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
15	0	0	0	6	9

TABLE OF CONTENTS

1. Introduction
2. Assessment summary
3. Test approach and methodology
4. Risk methodology
5. Scope
6. Assessment summary & findings overview
7. Findings & Tech Details
7.1 Unbounded global proposal loops can brick governance operations
7.2 Loss of user funds when creating non-finalized proposals with msg.value
7.3 Block gas limit dos can still happen via large single-transaction proposals
7.4 Lack of msg.value enforcement in proposal execution could lead to stranded assets
7.5 Single step ownership transfer process
7.6 Potential double-counting of xwell voting power in votingpoweraggregator
7.7 Unnecessary and unreachable code in proposal creation
7.8 No setter for _maxuserproposalcount
7.9 Cooldown grieving allows delaying redemption
7.10 Minimum quorum is not enforced in multichaingovernorv2
7.11 Fee-on-transfer or rebasing tokens can cause undercollateralization

7.12 Missing input validation

7.13 Typos

7.14 Open to-dos

7.15 Documentation mismatch in proposal cancellation states

1. INTRODUCTION

Moonwell engaged **Halborn** to perform a security assessment of their smart contracts from January 20th, 2026 to February 4th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The **Moonwell** codebase in scope consists of smart contracts implementing a multichain governance system with cross-chain proposal and voting aggregation, a modular voting power aggregator, and a staking mechanism for WELL tokens with cooldown and reward logic.

2. ASSESSMENT SUMMARY

Halborn was allocated 12 days for this engagement and assigned 2 full-time security engineers to conduct a comprehensive review of the smart contracts within scope. The engineers are experts in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, **Halborn** identified several areas for improvement to reduce the likelihood and impact of security risks, which were partially addressed by the **Moonwell team**. The main recommendations were:

- Implement a global cap on the number of live proposals, or refactor logic to avoid unbounded iteration.
- Implement validation requiring `msg.value == 0` when `finalize = false` to prevent permanent fund loss in creation of proposals.
- Add maximum action limit during `propose()`.

3. TEST APPROACH AND METHODOLOGY

Halborn conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (Foundry).
- Fork testing against main networks (Foundry).
- Static security analysis of scoped contracts, and imported functions (Slither).

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

Impact Metric (M_I)	Metric Value	Numerical Value
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (<i>r</i>)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (<i>s</i>)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

5. SCOPE

REPOSITORY ^

(a) Repository: `moonwell-contracts-v2`

(b) Assessed Commit ID: `93d240a`

(c) Items in scope:

- `src/governance/multichain/MultichainGovernorV2.sol`
- `src/governance/multichain/MultichainVoteCollectionV2.sol`
- `src/governance/multichain/VotingPowerAggregator.sol`
- `src/stkWell/StakedWellMoonbeam.sol`
- `moonwell-fi/moonwell-contracts-v2/commit/b350167e58f0803a1d24737d86fd2c52799ad795`

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID: ^

- `e12ea7a`
- `948001e`
- `991030d`

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	0	0	6	9

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
UNBOUNDED GLOBAL PROPOSAL LOOPS CAN BRICK GOVERNANCE OPERATIONS	LOW	RISK ACCEPTED - 02/10/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDATION DATE
LOSS OF USER FUNDS WHEN CREATING NON-FINALIZED PROPOSALS WITH MSG.VALUE	LOW	RISK ACCEPTED - 02/10/2026
BLOCK GAS LIMIT DOS CAN STILL HAPPEN VIA LARGE SINGLE-TRANSACTION PROPOSALS	LOW	RISK ACCEPTED - 02/10/2026
LACK OF MSG.VALUE ENFORCEMENT IN PROPOSAL EXECUTION COULD LEAD TO STRANDED ASSETS	LOW	RISK ACCEPTED - 02/10/2026
SINGLE STEP OWNERSHIP TRANSFER PROCESS	LOW	RISK ACCEPTED - 02/10/2026
POTENTIAL DOUBLE-COUNTING OF XWELL VOTING POWER IN VOTINGPOWERAGGREGATOR	LOW	SOLVED - 02/10/2026
UNNECESSARY AND UNREACHABLE CODE IN PROPOSAL CREATION	INFORMATIONAL	SOLVED - 02/10/2026
NO SETTER FOR _MAXUSERPROPOSALCOUNT	INFORMATIONAL	ACKNOWLEDGED - 02/10/2026
COOLDOWN GRIEFING ALLOWS DELAYING REDEMPTION	INFORMATIONAL	ACKNOWLEDGED - 02/10/2026
MINIMUM QUORUM IS NOT ENFORCED IN MULTICHAINGOVERNORV2	INFORMATIONAL	ACKNOWLEDGED - 02/10/2026
FEE-ON-TRANSFER OR REBASING TOKENS CAN CAUSE UNDERCOLLATERALIZATION	INFORMATIONAL	ACKNOWLEDGED - 02/10/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
MISSING INPUT VALIDATION	INFORMATIONAL	ACKNOWLEDGED - 02/10/2026
TYPOS	INFORMATIONAL	SOLVED - 02/10/2026
OPEN TO-DOS	INFORMATIONAL	SOLVED - 02/10/2026
DOCUMENTATION MISMATCH IN PROPOSAL CANCELLATION STATES	INFORMATIONAL	SOLVED - 02/10/2026

7. FINDINGS & TECH DETAILS

7.1 UNBOUNDED GLOBAL PROPOSAL LOOPS CAN BRICK GOVERNANCE OPERATIONS

// LOW

Description

The `MultichainGovernorV2` contract maintains a global set of live proposals (`_liveProposals`) and iterates over all of them in several critical functions, including `pause()` , `propose()` , `execute()` , and `_syncTotalLiveProposals()` . There is no global cap on the number of live proposals, only a per-user cap.

If many users create proposals simultaneously, the global set can grow large enough that these functions exceed block gas limits, causing them to revert and potentially bricking governance operations such as proposing, executing, canceling, or pausing.

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (3.3)

Recommendation

Implement a global cap on the number of live proposals, or refactor logic to avoid unbounded iteration. Alternatively, add mechanisms to allow for incremental cleanup or emergency recovery if the set becomes too large.

Remediation Comment

RISK ACCEPTED: The **Moonwell team** made a business decision to accept the risk of this finding and not alter the contracts.

7.2 LOSS OF USER FUNDS WHEN CREATING NON-FINALIZED PROPOSALS WITH MSG.VALUE

// LOW

Description

The `MultichainGovernorV2` contract's proposal creation mechanism exhibits an inconsistency in its handling of native tokens during multi-stage proposal drafting. When users initiate proposals with the `finalize` parameter set to `false` to create draft proposals for later completion, any native tokens sent via `msg.value` remain in the contract balance without utilization or recovery pathways. This creates a situation where participants in workflows can unintentionally and permanently lock assets within the contract.

 Copy Code

```
460 function propose(  
461     address[] memory targets,  
462     uint256[] memory values,  
463     bytes[] memory calldatas,  
464     string memory descriptionUri,  
465     bool finalize // - When false, msg.value is ignored  
466 ) external payable override whenNotPaused returns (uint256) {  
467  
468     uint256 proposalId = ++proposalCount;  
469     Proposal storage newProposal = proposals[proposalId];  
470  
471     // Store proposal data...  
472  
473     if (finalize) {  
474         // Only here is msg.value potentially used for execution  
475         _finalizeProposal(proposalId);  
476     }  
477  
478     return proposalId;  
479 }
```

The function lacks input validation to ensure that `msg.value == 0` when creating draft proposals (`finalize == false`). Since draft proposals do not execute any on-chain actions that would consume the sent ETH, any funds transmitted become permanently locked in the contract balance with no withdrawal or refund mechanism.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N (2.5)

Recommendation

Implement validation requiring `msg.value == 0` when `finalize = false` to prevent permanent fund loss in creation of proposals.

Remediation Comment

RISK ACCEPTED: The Moonwell team made a business decision to accept the risk of this finding and not alter the contracts.

7.3 BLOCK GAS LIMIT DOS CAN STILL HAPPEN VIA LARGE SINGLE-TRANSACTION PROPOSALS

// LOW

Description

The `MultichainGovernorV2` contract allows users to create proposals with an unlimited number of targets/calldatas in a single `propose()` call (when `finalize=true`). While the chunked proposal mechanism was introduced to mitigate block gas limit issues, **users can still call the function** by submitting all proposal data in one transaction, potentially creating:

1. **Un-executable proposals** that exceed block gas limits during execution.
2. **Storage bloat** from massive proposal structs.

The contract **lacks validation** to enforce maximum proposal action limits, making the chunking mechanism **optional rather than protective**.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

Add maximum action limit during `propose()` .

Remediation Comment

RISK ACCEPTED: The Moonwell team made a business decision to accept the risk of this finding and not alter the contracts.

7.4 LACK OF MSG.VALUE ENFORCEMENT IN PROPOSAL EXECUTION COULD LEAD TO STRANDED ASSETS

// LOW

Description

The `execute(uint256 proposalId)` function in the `MultichainGovernorV2` contract is intended to execute governance proposals, with the expectation that the sum of all `values` matches `msg.value` and that the contract's ETH balance remains unchanged after execution.

 Copy Code

```
551 /// @notice execute a proposal
552 /// can only be called if the proposal is in the succeeded state
553 /// can only be called when the contract is not paused
554 /// the sum of the values must be equal to the msg.value
555 /// the native token balance of this contract will remain unchanged before and after a proposal i
556 /// @param proposalId the id of the proposal to execute
557 function execute(
558     uint256 proposalId
559 ) external payable override whenNotPaused {
560     Proposal storage proposal = proposals[proposalId];
561
562     ProposalState proposalState = state(proposalId);
563     if (proposalState != ProposalState.Succeeded) {
564         revert InvalidProposalState(proposalState, ProposalState.Succeeded);
565     }
566     if (
567         block.timestamp >
568         proposal.crossChainVoteCollectionEndTimestamp + _executionWindow
569     ) {
570         revert ProposalExpired();
571     }
572
573     proposal.executed = true;
574
575     /// remove the proposal that is about to be executed from all proposals,
576     /// and remove from inactive proposals from user list
577     _syncTotalLiveProposals();
578
579     unchecked {
580         for (uint256 i = 0; i < proposal.targets.length; i++) {
581             _functionCallWithValue(
582                 proposal.targets[i],
583                 proposal.calldatas[i],
584                 proposal.values[i]
585             );
586         }
587     }
588
589     emit ProposalExecuted(proposalId);
590 }
```

However, the function does not validate that `msg.value` equals the sum of the proposal's `values`, nor does it refund excess ETH or prevent execution using the contract's existing ETH balance. As a result, any externally owned account (EOA) can send excess ETH that becomes permanently trapped, or execute proposals using ETH already held by the contract, breaking the documented invariant and potentially draining funds that may have been intended for other purposes.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N (2.5)

Recommendation

Add a check to ensure that `msg.value` matches the sum of the proposal's `values` and refund any excess ETH.

Remediation Comment

RISK ACCEPTED: The Moonwell team made a business decision to accept the risk of this finding and not alter the contracts.

7.5 SINGLE STEP OWNERSHIP TRANSFER PROCESS

// LOW

Description

The `VotingPowerAggregator` contract inherits the `OwnableUpgradeable` contract implementation from OpenZeppelin's library and is used to restrict access to certain functions to the contract owner. The `Ownable` pattern allows the contract owner to transfer ownership to another address using the `transferOwnership()` function. However, the `transferOwnership()` function does not include a two-step process to transfer ownership.

It is crucial that the address to which ownership is transferred is verified to be active and willing to assume ownership responsibilities. Otherwise, the contract could be locked in a situation where it is no longer possible to make administrative changes to it.

Additionally, the `renounceOwnership()` function allows renouncing to the owner permission. Renouncing ownership before transferring it would result in the contract having no owner, eliminating the ability to call privileged functions.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (2.0)

Recommendation

Consider using OpenZeppelin's `Ownable2StepUpgradeable` contract over the `OwnableUpgradeable` implementation. `Ownable2StepUpgradeable` provides a two-step ownership transfer process, which adds an extra layer of security to prevent accidental ownership transfers.

Additionally, it is recommended that the owner cannot call the `renounceOwnership()` function to avoid losing ownership of the contract.

Remediation Comment

RISK ACCEPTED: The Moonwell team made a business decision to accept the risk of this finding and not alter the contracts.

7.6 POTENTIAL DOUBLE-COUNTING OF XWELL VOTING POWER IN VOTINGPOWERAGGREGATOR

// LOW

Description

The `VotingPowerAggregator` contract aggregates voting power from both the `xWELL` token and a configurable set of snapshot sources. However, there is no safeguard preventing the owner from adding the `xWELL` contract itself to the `_snapshotSources` set.

If this occurs, voting power from `xWELL` will be counted twice—once via the snapshot interface and once via the direct `xWell.getVotes()` call—leading to inflated voting power for all users. This undermines the accuracy of governance voting tallies.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:C/D:N/Y:N (2.0)

Recommendation

Add a check in `addSnapshotSource()` to prevent the `xWELL` contract address from being added to `_snapshotSources`. Alternatively, document this invariant and ensure governance processes enforce it.

Remediation Comment

SOLVED: The **Moonwell team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/e12ea7a2554cd58cdaa0c6845af088e027e0fd70>

7.7 UNNECESSARY AND UNREACHABLE CODE IN PROPOSAL CREATION

// INFORMATIONAL

Description

In the `MultichainGovernorV2` contract's `propose` function, the code performs redundant validation checks that can never actually fail, resulting in unnecessary gas consumption and misleading code paths. The issue stems from the contract checking whether a newly auto-incremented proposal ID already exists in `EnumerableSets`, a condition that is mathematically impossible given the proposal ID generation mechanism.

 Copy Code

```
321 | if (!_userLiveProposals[msg.sender].add(proposalId)) {  
322 |     revert ProposalAlreadyExists();  
323 | }  
324 | if (!_liveProposals.add(proposalId)) {  
325 |     revert ProposalAlreadyExists();  
326 | }
```

When a user creates a proposal, the contract increments `proposalCount` and assigns this new unique value to `proposalId`, guaranteeing that this specific numeric identifier has never been used before in the system's history. Immediately after this guaranteed-unique generation, the code attempts to add this ID to both user-specific and global proposal sets while checking the return value of the `add()` operation, which would only return false if the ID already existed in those sets. Since the ID was just freshly minted and couldn't possibly have been added previously, these conditional checks will always evaluate to true, making the associated `ProposalAlreadyExists` error revert paths completely unreachable.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

Remove the unnecessary checks.

Remediation Comment

SOLVED: The **Moonwell team** solved this finding by removing unnecessary checks.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/948001ec2dd304fcedddd59acecace12c932a676>

7.8 NO SETTER FOR _MAXUSERPROPOSALCOUNT

// INFORMATIONAL

Description

The `MultichainGovernorV2::_maxUserProposalCount` variable is hardcoded to `2` during initialization with no governance setter function, preventing future adjustments if the limit needs to change.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

Add a setter function to change `_maxUserProposalCount`.

Remediation Comment

ACKNOWLEDGED: The Moonwell team made a business decision to acknowledge this finding and not alter the contracts.

7.9 COOLDOWN GRIEFING ALLOWS DELAYING REDEMPTION

// INFORMATIONAL

Description

The `StakedTokenV2.stake()` and `_transfer()` functions allow anyone to stake tokens on behalf of another user or transfer stkWELL tokens to them, which triggers the `getNextCooldownTimestamp()` function. This computes a weighted average of cooldown timestamps, potentially shifting the recipient's cooldown forward if they already have an active one.

While the recipient gains stkWELL balance, their redemption is delayed, forcing extra waiting time. This is a griefing vector where an attacker spends underlying tokens to inconvenience a victim without gaining profit.

For example:

1. Victim stakes tokens and activates cooldown (e.g., `stakersCooldowns[victim] = currentTimestamp`).
2. Victim waits until just before the cooldown ends, becoming redeemable.
3. Attacker stakes tokens on behalf of the victim (via `stake(victim, amount)`) or transfers stkWELL to them (via `transfer(victim, amount)`).
4. The weighted average in `getNextCooldownTimestamp()` pushes the victim's cooldown timestamp later.
5. Victim's immediate `redeem()` reverts with "INSUFFICIENT_COOLDOWN," requiring extra wait.

BVSS

AO:A/AC:H/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (1.6)

Recommendation

Consider restricting third party staking.

Alternatively, document in user facing materials and contract comments that incoming transfers or stakes during an active cooldown may delay redemption due to the weighted average cooldown mechanism.

Remediation Comment

ACKNOWLEDGED: The Moonwell team made a business decision to acknowledge this finding and not alter the contracts.

7.10 MINIMUM QUORUM IS NOT ENFORCED IN MULTICHAINGOVERNORV2

// INFORMATIONAL

Description

The `MultichainGovernorV2` contract allows the quorum value to be set to zero via the `updateQuorum()` and internal `_setQuorum()` functions.

This means governance proposals could undermine the integrity of the voting process. The intended behavior is for proposals to require a minimum number of votes to pass, but the current implementation only enforces a maximum quorum.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:N/Y:N (1.5)

Recommendation

Enforce a nonzero minimum quorum in `_setQuorum()`, or document the risk and ensure governance processes prevent setting quorum to zero.

Remediation Comment

ACKNOWLEDGED: The **Moonwell team** made a business decision to acknowledge this finding and not alter the contracts.

7.11 FEE-ON-TRANSFER OR REBASING TOKENS CAN CAUSE UNDERCOLLATERALIZATION

// INFORMATIONAL

Description

The `StakedTokenV2.stake()` function mints stkWELL tokens to the user based on the input `amount` and then calls `STAKED_TOKEN.safeTransferFrom(msg.sender, address(this), amount)`. If the underlying `STAKED_TOKEN` is a fee on transfer or rebasing token, the contract may receive fewer tokens than the amount of stkWELL minted, resulting in under collateralization.

This could lead to situations where the contract does not have enough underlying tokens to fulfill redemptions for all stkWELL holders. A similar issue exists in `claimRewards()`, which assumes the full `amount` of `REWARD_TOKEN` is transferred.

BVSS

AO:S/AC:M/AX:L/R:N/S:U/C:N/A:N/I:N/D:H/Y:N (1.0)

Recommendation

Document that only standard, non-deflationary ERC20 tokens should be used as `STAKED_TOKEN` and `REWARD_TOKEN`.

Alternatively, update the staking and reward logic to mint stkWELL and distribute rewards based on the actual amount received by the contract, not the input amount.

Remediation Comment

ACKNOWLEDGED: The Moonwell team made a business decision to acknowledge this finding and not alter the contracts.

7.12 MISSING INPUT VALIDATION

// INFORMATIONAL

Description

Throughout the codebase, there are several instances where input values are assigned without proper validation. For example, ensuring that an input address is not the zero address or that an integer falls within a valid range. Failing to validate inputs before assigning them to state variables can lead to unexpected system behavior or even complete failure.

Instances of this issue include:

- In `StakedWellMoonbeam.setCooldownSeconds()`, `cooldownSeconds` is not checked to fall within a valid threshold.
- In `StakedWellMoonbeam.setUnstakeWindow()`, `unstakeWindow` is not checked to fall within a valid threshold.

BVSS

AO:S/AC:L/AX:M/R:F/S:U/C:N/A:H/I:N/D:H/Y:N (0.3)

Recommendation

All administrative setter functions shall be validated by `require` checks that enforce the defined `MIN` and `MAX` thresholds and ensure non-zero addresses where applicable, with explicit revert messages. The `cooldownSeconds` and `unstakeWindow` values shall be constrained to safe ranges prior to assignment. Corresponding constants and `NatSpec` documentation shall be provided, `events` shall be emitted upon successful updates, and unit tests shall be implemented to verify that out-of-range values revert as expected.

Remediation Comment

ACKNOWLEDGED: The **Moonwell team** made a business decision to acknowledge this finding and not alter the contracts.

7.13 TYPOS

// INFORMATIONAL

Description

Throughout the codebase, there are several instances of typos in comments. While these typos do not affect the functionality of the code, they can make the codebase harder to read and understand. It is recommended to fix these typos to improve the readability of the codebase.

Instances of this issue include:

- The word **MOONBEAM** is misspelled as **MOOMBEAM** in the **MultichainVoteCollectionV2** contract.
- The word **GOVERNOR** is misspelled as **GOVERNER** in the **MultichainVoteCollectionV2** contract.
- The word **Quorum** is misspelled as **Quroum** in the **QuroumVotesChanged** event of the **IMultichainGovernorV2** interface.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to fix all typos to improve the readability of the codebase.

Remediation Comment

SOLVED: The **Moonwell team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/991030ddc0d06b30bcd658bae5e572b0d27ff5cd>

7.14 OPEN TO-DOS

// INFORMATIONAL

Description

In the **MultichainGovernorV2** contract, there are several open TO-DO comments.

Open TO-DO comments should be addressed before the final release of the project, as these may indicate that the functionality has not been not completely implemented.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Ensure that all TO-DO comments are addressed before the final release of the project.

Remediation Comment

SOLVED: The **Moonwell team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/991030ddc0d06b30bcd658bae5e572b0d27ff5cd>

7.15 DOCUMENTATION MISMATCH IN PROPOSAL CANCELLATION STATES

// INFORMATIONAL

Description

The documentation for the `cancel()` function in `MultichainGovernorV2` states that proposals can be canceled in the succeeded or cross-chain vote collection states. However, the implementation only allows cancellation in the `Active` or `Init` states. This discrepancy may confuse users or integrators about when proposals are eligible for cancellation.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Update the documentation to match the implementation, or adjust the code to allow cancellation in the documented states.

Remediation Comment

SOLVED: The **Moonwell team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/991030ddc0d06b30bcd658bae5e572b0d27ff5cd>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.